

Table of Contents

- 1 📖 핸드북 메인 로드맵
- 2 🏠 시스템 아키텍처
- 3 📋 직무 역량 계획서
- 4 🧠 지식 체계 문서
- 5 🔧 Phase 0 - Agent Loop 튜토리얼
 - 5.1 Overview
 - 5.2 v0 - Bash is All You Need
 - 5.3 v1 - Model as Agent
 - 5.4 v2 - Structured Planning
 - 5.5 v3 - Subagent Mechanism
 - 5.6 v4 - Skills Mechanism
- 6 ⚙️ 스킬 & 레퍼런스
 - 6.1 Agent Builder Skill
 - 6.2 Agent Philosophy
 - 6.3 Code Review Skill
 - 6.4 MCP Builder Skill
 - 6.5 PDF Handling Skill

1 Agentic Engineering Handbook (에이전트 엔지니어링 핸드북)

OpenAI, Anthropic, Google, MCP, Harness, Evals 및 프로덕션 에이전트 시스템 구축을 위한 가이드 및 학습 로드맵

License MIT

Last Updated 2026-07-25

1.1 💡 핸드북 소개 (Why This Repository?)

AI 산업은 본격적인 **에이전트 시대(Agentic Era)**로 진입했습니다. 프로덕션 수준의 AI 시스템을 구축하려면 에이전트 루프, 도구 활용(Tool Use), MCP(Model Context Protocol), 메모리 관리, 장기 실행 워크플로우, 코딩 에이전트, 에이전트 하네스(Harness), 평가(Evals), 그리고 안전성(Safety)에 대한 깊은 이해가 필수적입니다.

그러나 이러한 핵심 기술 지식은 OpenAI 블로그, Anthropic 엔지니어링 아티클, SDK 공식 문서, 기술 서적 및 논문 등으로 파편화되어 존재합니다.

본 핸드북은 **179개의 엄선된 기술 자료와 아티클**을 하나의 체계적인 학습 로드맵으로 정리한 가이드입니다.

최종 목표: 세계적인 수준의 에이전트 엔지니어(Agentic Engineer)로 성장하기

1.2 📖 핸드북 활용 가이드 (How To Use This Handbook)

자신의 학습 목표와 출발점에 맞춰 아래 가이드를 활용하세요:

- **에이전트를 처음 접하는 경우:** **학습 로드맵**의 Phase 0부터 Phase 6까지 순서대로 진행하세요. 각 단단계의 **필독** (Read First), **심화** (Then Read), 및 **실습 과제** (Build Exercise) 를 체크리스트처럼 활용하세요.
- **이미 LLM 애플리케이션을 개발해 본 경우:** **Phase 2 – MCP & 톨 생태계** 또는 **Phase 3 – 컨텍스트, 메모리 & 스킬**부터 시작하여 에이전트 루프, 도구 호출, Evals 및 프로덕션 설계 공백을 메우세요.
- **실전 프로젝트를 구축하려는 경우:** 각 Phase별 **실습 과제** (Build Exercise) 프롬프트를 활용한 후, 코딩 에이전트, 보안, 코드 리뷰, SRE 등을 다루는 **실전 응용 트랙**으로 확장하세요.
- **참고 자료를 찾는 경우:** **전체 레퍼런스 리스트**로 바로 이동하세요. 핵심 개념은 **P0**, 실무 구현 상세는 **P1**, 심화 배경 지식은 **P2** 로 구분되어 있습니다.

1.3 🎯 학습 로드맵 (Learning Roadmap)

1.3.1 Phase 0 – 에이전트 루프 바닥부터 구현하기 (Agent Loop From Scratch)

Claude Code나 자율 AI 코딩 도구를 다루다 보면 파일 읽기, 명령 실행, 코드 수정, 서브에이전트 위임 등이 마치 마법처럼 동작하는 것처럼 느껴집니다.

하지만 엔지니어링 관점에서 에이전트의 핵심 구조는 매우 단순합니다:

$$\text{\text{Model}} + \text{\text{Tools}} + \text{\text{One Loop}}$$

이 핵심 루프를 직접 구현해 보면 에이전트 시스템 전체를 훨씬 명확하게 이해할 수 있습니다:

- 에이전트가 언제 먼저 계획(Plan)을 세우고, 언제 즉시 실행(Act)해야 하는가?
- 명시적인 To-Do 리스트가 장기 작업 실행 시 에이전트의 이탈(Drift)을 방지하는 이유
- 서브에이전트(Subagent)가 주 컨텍스트(Main Context)를 보호하면서 탐색 영역을 넓히는 원리
- 스킬(Skills), MCP, 훅(Hooks)이 동일한 기본 루프 위에서 어떻게 기능을 확장하는가?

본 단계는 [shareAI-lab/mini-claude-code](#)의 오픈소스 튜토리얼을 기반으로 한국어 스터디 노트를 추가하여 구성되었습니다.

단계	한국어 튜토리얼 문서	구현 코드
v0	Bash 하나면 충분하다 (Bash is All You Need)	v0_bash_agent.py
v1	에이전트로서의 모델 (Model as Agent)	v1_basic_agent.py
v2	구조화된 계획 수립 (Structured Planning)	v2_todo_agent.py
v3	서브에이전트 메커니즘 (Subagent Mechanism)	v3_subagent.py
v4	스킬 메커니즘 (Skills Mechanism)	v4_skills_agent.py

1.3.2 Phase 1 – 에이전트 기초 (Agent Foundations)

워크플로우 vs 에이전트, 톨 루프, 핸드오프, 가드레일에 대한 공통 용어와 멘탈 모델을 정립합니다.

핵심 멘탈 모델

에이전트를 도입해야 할까? (Barry Zhang, Anthropic의 4가지 질문 체크리스트)

질문	No 인 경우 → 워크플로우(Workflow)	Yes 인 경우 → 에이전트(Agent)
작업이 충분히 복잡한가?	결정 트리(Decision Tree)로 완벽히 매핑 가능	불확실하고 모호한 문제 공간

질문	No 인 경우 → 워크플로우(Workflow)	Yes 인 경우 → 에이전트(Agent)
작업의 가치가 충분한가?	실행당 < \$0.10 비용	실행당 > \$1 이상 가치 생성
핵심 능력을 모델이 잘 수행하는가?	약한 고리가 전체 체인을 단절시킴	모델이 모든 단계를 안정적으로 처리
오류 비용이 낮고 감지 가능한가?	높은 오류 비용 + 감지 어려움 → 사람 개입 필요	테스트/CI를 통해 오류 자동 감지 가능

에이전트의 관점에서 생각하라 (Think like the agent): 대부분의 설계 실패는 사람의 시각에서 시스템을 설계하기 때문에 발생합니다. 에이전트의 컨텍스트 윈도우 내부(시스템 프롬프트 + 도구 설명 + 최근 관찰된 결과) 입장이 되어 보세요. 각 단계에서 에이전트가 올바르게 행동할 수 있는 충분한 정보가 주어져 있는지 점검해야 합니다.

→ 참고 영상: [How We Build Effective Agents](#)

필독 아티클 (Read First)

#	제목	제공
1	System Prompts	Anthropic
2	Prompt guidance	OpenAI
3	Function Calling	OpenAI
4	Tool use overview	Anthropic
5	Function calling - Gemini API	Google
6	Building effective agents	Anthropic
7	New tools for building agents	OpenAI
8	Agents SDK overview	OpenAI

1.3.3 Phase 2 – MCP & 툴 생태계 (MCP & Tool Ecosystem)

MCP 서버/클라이언트 구조, 로컬 vs 원격 연결, 도구 동적 로딩, 승인 절차 및 커넥터 경계를 습득합니다.

필독 아티클 (Read First)

#	제목	제공
1	Introducing the Model Context Protocol	Anthropic
2	MCP and Connectors	OpenAI
3	Building MCP servers for ChatGPT Apps and API integrations	OpenAI

1.3.4 Phase 3 – 컨텍스트, 메모리 & 스킬 (Context, Memory & Skills)

컨텍스트 윈도우 제어, 단기/장기 메모리 관리, 스킬/플러그인 작성법 및 AGENTS.md 활용법을 익힙니다.

필독 아티클 (Read First)

#	제목	제공
1	Agent Skills Specification	Agent Skills
2	Effective context engineering for AI agents	Anthropic
3	The new rules of context engineering for Claude 5 generation models	Anthropic
4	Building Reliable Agents with Memory and Compaction	OpenAI

1.3.5 Phase 4 – 에이전트 하네스 & 장기 실행 에이전트 (Harness & Long-Running Agents)

이벤트 스트림, 스레드 관리, 도구 실행 샌드박스, 상태 복구, 인간 승인 루프를 구현하는 에이전트 하네스 기술을 마스터합니다.

필독 아티클 (Read First)

#	제목	제공
1	Unrolling the Codex agent loop	OpenAI
2	Unlocking the Codex harness: how we built the App Server	OpenAI
3	Effective harnesses for long-running agents	Anthropic

1.3.6 Phase 5 – 코딩 & 워크스페이스 에이전트 (Coding & Workspace Agents)

Codex와 Claude Code 제품/SDK 구조를 비교하고, 대규모 코드베이스에서의 멀티 에이전트 협업 및 IDE 연동 기법을 다룹니다.

필독 아티클 (Read First)

#	제목	제공
1	AGENTS.md	Agentic AI Foundation
2	Introducing Codex	OpenAI
3	Best practices for Claude Code	Anthropic
4	How Claude Code works in large codebases	Anthropic

1.3.7 Phase 6 – 평가, 안전성 & 프로덕션 (Evals, Safety & Production)

사전/사후 평가 루프, 트레이싱(Tracing), 프롬프트 인젝션 방어, 권한 제어 및 회귀 테스트 모니터링 체계를 구축합니다.

필독 아티클 (Read First)

#	제목	제공
1	Demystifying evals for AI agents	Anthropic
2	The six generations of AI agents and how to eval them	Braintrust
3	Agent observability powers agent evaluation	LangChain

1.4 🧩 실전 응용 트랙 (Applied Practice Tracks)

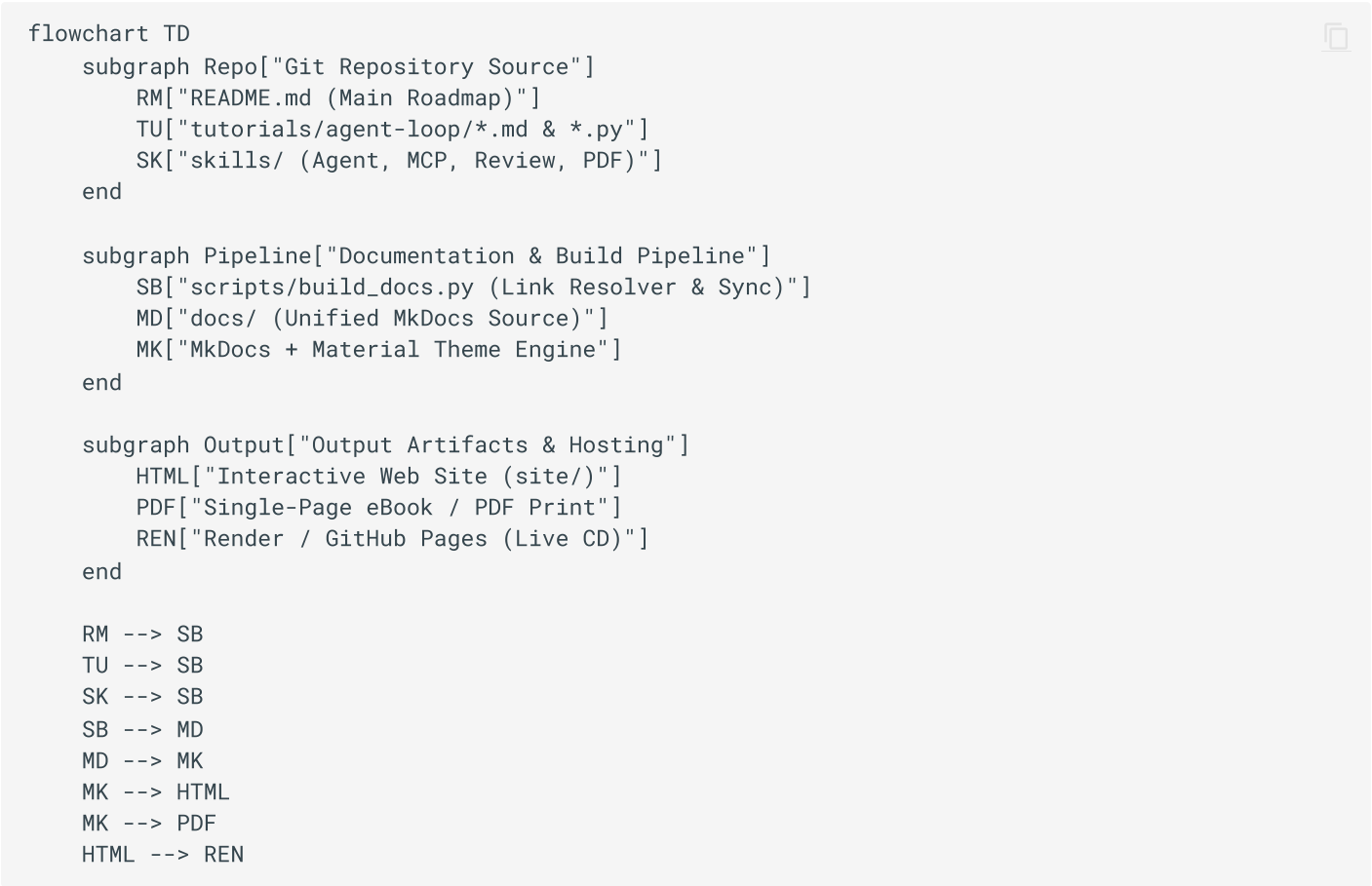
트랙 명칭	추천 시작 문서	중요성 및 핵심 목표
에이전트 코딩 워크 플로우	Coding Agents 101, Claude Code Best Practices	에이전트 이론을 명확한 프롬프팅, 검증 체계, 병렬 작업 등 일상적인 코딩 습관으로 전환합니다.
명세 기반 시스템 구축	The spec is dead, long live the spec!	명세서(Spec)와 프롬프트를 에이전트가 즉시 실행 가능한 소스 코드로 다룹니다.
컨텍스트 실패 모드 분석	How Long Contexts Fail, Context Rot	컨텍스트 오염, 환각, 검색 과부하 및 성능 저하 원인을 정확히 진단합니다.
Evals & 관측 가능 성	Demystifying evals for AI agents	오프라인/온라인 평가, 데이터셋 정제, 회귀 게이트 구축 피드백 루프를 만듭니다.
딥리서치 에이전트	Deep research, Open Deep Research	장기 실행 탐색 에이전트: 계획, 검색, MCP, 출처 인용 및 보고서 합성 기법을 연습합니다.
에이전트 보안 & 가 드레일	OWASP Top Ten, Prompt Injection Analysis	프롬프트 인젝션, 권한 오용, 샌드박스 등 에이전트 보안 베스트 프랙티스를 적용합니다.

2 Agentic Engineering Handbook — Architecture & System Design

2.1 1. System Overview

The **Agentic Engineering Handbook** system is structured into two core layers: 1. **Core Agent Systems & Tutorials Layer**: Contains hands-on implementations of LLM agent loops (`v0` to `v4`), skill suites, MCP integrations, and architectural patterns. 2. **Interactive Publishing & Documentation Pipeline**: Converts raw Markdown guides and code tutorials into a production-grade interactive documentation site and printable PDF eBook via MkDocs Material and Render static site hosting.

2.2 2. System Architecture Diagram



2.3 3. Component Details

2.3.1 3.1 Agent Loop Evolution (`tutorials/agent-loop/`)

Version	Component Name	Description	Key Modules
v0	Bash Agent Loop	Minimal zero-dependency bash execution loop	v0_bash_agent.py
v1	Model as Agent	LLM tool selection and dynamic invocation	v1_basic_agent.py
v2	Structured Planning	Plan-and-execute loop with todo list management	v2_todo_agent.py
v3	Subagent Orchestration	Task decomposition and parallel subagent spawning	v3_subagent.py
v4	Skills System	Modular skill injection, dynamic tool loading	v4_skills_agent.py

2.3.2 3.2 Documentation Pipeline (scripts/)

- scripts/build_docs.py : Resolves relative Markdown links, copies root files to docs/ , and generates navigation indexes.
- scripts/start_docs.py : Provides 1-click local server binding (0.0.0.0:8000) for cross-device & mobile viewing on local Wi-Fi.
- mkdocs.yml : Production theme config supporting light/dark themes, code copying, search auto-complete, and mkdocs-print-site-plugin.

2.4 4. Deployment Pipeline (Render / GitHub Pages)

- **Infrastructure:** Static Site Hosting on Render.com (render.yaml).
- **Build Command:** pip install -r requirements.txt && python scripts/build_docs.py && mkdocs build
- **Publish Path:** ./site

3 Agentic Engineer 직무 역량 및 성장 계획서 (Career Plan)

3.1 1. 직무 정의 (Role Definition)

Agentic Engineer (에이전트 엔지니어)는 단순한 단발성 LLM 프롬프트 작성을 넘어, 자율적 문제 해결이 가능한 **자율 AI 에이전트 시스템**, **MCP(Model Context Protocol)** 툴링, 컨텍스트 파이프라인, 에이전트 평가(Evals) 및 프로덕션 하네스(Harness)를 설계·구축·운영하는 최고 수준의 AI 엔지니어입니다.

3.2 2. 로드맵 단계별 역량 모델 (Competency Framework)

단계	주요 영역	핵심 보유 역량 (Key Skills)	실행 과제 (Build Exercises)
Phase 0	Agent Loop Base	LLM 기본 루프, Bash 실행기, 도구 호출, 파일 입출력	v0 ~ v4 에이전트 루프 바닥부터 구현하기
Phase 1	Agent Foundations	프롬프트 엔지니어링, 시스템 프롬프트 작성, 에이전트 판단 프레임워크	Anthropic / OpenAI 에이전트 베스트 프랙티스 습득
Phase 2	MCP & Tool Ecosystem	Model Context Protocol 서버/클라이언트 개발, 커스텀 툴 샌드박스	MCP Server 구축 및 Claude/Antigravity 연동
Phase 3	Context, Memory & Skills	장기/단기 메모리 윈도우 관리, 요약 기법, 스킬 모듈화	Antigravity/Claude 스킬 세트 구현 및 주입 테스트
Phase 4	Agent Harness & Multi-Agent	에이전트 오케스트레이션, 서브에이전트 분기, 상태 분리	멀티에이전트 협업 루프 개발
Phase 5	Evals & Benchmarks	SWE-bench, 에이전트 성능 측정, 벤치마크 설계, 회귀 테스트	Custom Eval Harness 구축 및 에이전트 정량 평가
Phase 6	Security & Production	샌드박스, 가드레일, 안전성 검증, 불확실성 제어	프로덕션 레벨 에이전트 배포 및 가드레일 적용

3.3 3. 핵심 마일스톤 및 실행 계획

- 1. 기반 기술 내재화 (Phase 0~2)
- 2. tutorials/agent-loop/ 코드의 백그라운드 원리 파악 및 커스텀 에이전트 제작

3. MCP Server 생태계 이해 및 사내/개인 도구의 MCP 모듈화
4. 에이전트 오케스트레이션 및 스킬화 (Phase 3~4)
5. 에이전트가 사용할 도구, 지식(Knowledge Items), 스킬(Skills) 표준 정의 및 모듈화
6. 프로덕션 검증 및 평가 체계 구축 (Phase 5~6)
7. 자동화된 Evals 구축으로 에이전트의 코드 생성 및 문제 해결 성공률 측정
8. Render / Cloud 인프라 기반 정적/동적 에이전트 서비스 배포 자동화

4 Agentic Engineering Handbook — Knowledge System Document

4.1 1. 개요 (Overview)

본 지식 문서는 **OpenAI, Anthropic, Google, MCP, Harness, Evals, 프로덕션 에이전트 시스템**에 관한 179개의 엄선된 리소스와 학습 자료를 체계적으로 분류한 지식 맵(Knowledge Map)입니다.

4.2 2. 지식 영역 분류 (Knowledge Domain Taxonomy)

4.2.1 Domain A. Agent Loops & Reasoning Patterns

- **Model + Tools + Loop**: 기본 루프 구성 요소 및 컨텍스트 제어
- **Planning Mechanisms**: ReAct, Plan-and-Solve, Todo 관리, 구조화된 계획 수립
- **Subagent & Delegation**: 주 컨텍스트 보호 및 서브에이전트 태스크 위임 패턴

4.2.2 Domain B. Protocols & Tool Interfaces

- **MCP (Model Context Protocol)**: 표준 툴 인터페이스, 서버/클라이언트 아키텍처
- **Function Calling**: LLM 별 (OpenAI, Gemini, Claude) 도구 스키마 정의 및 파싱 베스트 프랙티스

4.2.3 Domain C. Context, Memory & Skills

- **Context Compaction**: 롱 컨텍스트 윈도우 관리 및 요약 기법
- **Memory Architectures**: Ephemeral vs Persistent 메모리, 에피소딕 메모리
- **Skills System**: 런타임 스킬 주입 및 YAML 기반 스킬 템플릿

4.2.4 Domain D. Evals & Benchmarks

- **SWE-bench**: 에이전트 코딩 능력 평가 벤치마크
 - **Agent Evaluation Frameworks**: 정량적 성공률, 정밀도, 재현율 측정 도구
-

4.3 3. 웹 및 전자책 출판 연동

본 지식 문서 및 전체 리소스는 [mkdocs.yml](#) 설정과 Render 배포 파이프라인을 통해 웹 및 PDF 전자책 형태로 실시간으로 통합 및 동기화됩니다.

5 🛠️ Phase 0 - Agent Loop 튜토리얼

5.1 Phase 0 – Agent Loop Tutorials

This section covers building an agent loop from scratch across v0 to v4.

5.2 v0: Bash 하나면 충분하다 (Bash is All You Need)

궁극의 단순화: 단 50줄의 코드, 1개의 도구로 완성하는 완전한 에이전트.

v1, v2, v3 에이전트를 만들기 전에 본 튜토리얼에서는 에이전트의 본질이 무엇인지 질문을 던집니다.

v0는 불필요한 레이어를 제거하고 오직 핵심만 남김으로써 이에 대한 답을 제시합니다.

5.2.1 핵심 통찰 (The Core Insight)

유닉스(Unix) 철학: 모든 것은 파일이며, 모든 것은 파이프(pipe)로 연결될 수 있다. Bash는 이 세계로 통하는 관문입니다:

요구사항	Bash 명령 대응
파일 읽기	<code>cat</code> , <code>head</code> , <code>grep</code>
파일 쓰기	<code>echo '...' > file</code>
코드/파일 탐색	<code>find</code> , <code>grep</code> , <code>rg</code>
실행 및 테스트	<code>python</code> , <code>npm</code> , <code>make</code>
서브에이전트 위임	<code>python v0_bash_agent.py "작업 설명"</code>

마지막 줄이 가장 중요한 통찰입니다: **Bash를 통해 자기 자신을 다시 호출하면 서브에이전트(Subagent)가 구현됩니다.** 별도의 Task 도구나 에이전트 레지스트리 없이 재귀 호출만으로 완벽한 서브에이전트가 완성됩니다.

5.2.2 완전한 코드 (The Complete Code)

```
#!/usr/bin/env python
from anthropic import Anthropic
import subprocess, sys, os

client = Anthropic(api_key="your-key", base_url="...")
TOOL = [{
    "name": "bash",
    "description": """Execute shell command. Patterns:
- Read: cat/grep/find/ls
- Write: echo '...' > file
- Subagent: python v0_bash_agent.py 'task description'""",
    "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required":
["command"]}
}]
SYSTEM = f"CLI agent at {os.getcwd()}. Use bash. Spawn subagent for complex tasks."

def chat(prompt, history=[]):
    history.append({"role": "user", "content": prompt})
    while True:
        r = client.messages.create(model="...", system=SYSTEM, messages=history, tools=TOOL,
max_tokens=8000)
        history.append({"role": "assistant", "content": r.content})
        if r.stop_reason != "tool_use":
            return "".join(b.text for b in r.content if hasattr(b, "text"))
        results = []
        for b in r.content:
            if b.type == "tool_use":
                out = subprocess.run(b.input["command"], shell=True, capture_output=True,
text=True, timeout=300)
                results.append({"type": "tool_result", "tool_use_id": b.id, "content": out.stdout
+ out.stderr})
        history.append({"role": "user", "content": results})

if __name__ == "__main__":
    if len(sys.argv) > 1:
        print(chat(sys.argv[1])) # 서버에이전트 모드
    else:
        h = []
        while (q := input(">> ")) not in ("q", ""):
            print(chat(q, h))
```

5.2.3 요약

1. 단 하나의 도구(Bash)로 모든 입출력 및 외부 명령을 통합 처리합니다.
2. 자기 자신 재귀 호출로 서버에이전트의 컨텍스트 분리를 달성합니다.
3. 에이전트 루프의 본질은 결국 "모델 + 도구 + 무한 루프"의 결합입니다.

← 메인 로드맵으로 돌아가기

5.3 v1: 에이전트로서의 모델 (Model as Agent)

기본 툴 사용 루프와 동적 도구 호출 메커니즘

v1 단계에서는 LLM이 단순 텍스트 생성을 넘어, 스스로 판단하여 내장된 도구(File Operations, Bash execution 등)를 선택하고 실행한 후 결과를 관찰(Observe)하는 기본 에이전트 루프를 만듭니다.

5.3.1 핵심 개념 (Core Concepts)

- 도구 정의 (Tool Definition):** JSON Schema 형태로 에이전트가 수행 가능한 도구 목록을 모델에 전달합니다.
- 이행 제어 (Tool Use Decision):** 모델이 `stop_reason == "tool_use"` 를 반환할 때, 에이전트 코드가 이를 수신하고 해당 함수를 실행합니다.
- 관찰 피드백 (Observation Feedback):** 실행 결과를 `tool_result` 타입으로 히스토리에 누적시켜 모델이 다음 행동을 결정하게 합니다.

5.3.2 에이전트 실행 루프 구조

```
sequenceDiagram
    participant User as 사용자
    participant Agent as Agent Loop
    participant LLM as Claude / LLM
    participant System as 시스템 / Tools

    User->>Agent: 작업 요청 (Prompt)
    loop Tool Loop
        Agent->>LLM: Messages + Tools Schema
        LLM-->>Agent: Action (Tool Call Response)
        alt stop_reason == "tool_use"
            Agent->>System: Execute Tool (e.g. read_file, bash)
            System-->>Agent: Output / Result
            Agent->>Agent: Append Tool Result to History
        else stop_reason == "end_turn"
            Agent-->>User: Final Response
        end
    end
end
```

5.3.3 주요 구현 요약 (`v1_basic_agent.py`)

- `read_file`, `write_file`, `edit_file`, `bash` 등의 세분화된 전용 도구 구현
- 대화 히스토리 추적 및 오류 예외 처리
- 인라인 코드 실행 결과를 관찰 피드백으로 루프 재진입

[← 메인 로드맵으로 돌아가기](#) | [다음: v2 구조화된 계획 수립](#) →

5.4 v2: 구조화된 계획 수립 (Structured Planning)

To-Do 리스트 관리와 에이전트 목표 이탈(Drift) 방지

작업이 복잡해지고 대화 길이가 길어질수록 에이전트는 원래 목표를 잊거나 엉뚱한 방향으로 탐색을 계속하는 **목표 이탈(Goal Drift)** 현상을 겪게 됩니다.

v2 단계에서는 명시적인 **Todo 도구 및 계획 수립(Plan-and-Execute)** 메커니즘을 추가하여 복잡한 작업을 안정적으로 완수하도록 설계합니다.

5.4.1 핵심 메커니즘 (Key Mechanisms)

1. Todo 도구 도입:

2. `create_todo`: 작업 세부 목표 및 상태 관리

3. `update_todo`: 완료(completed), 진행중(in_progress), 보류(pending) 상태 갱신

4. 상태 가시성 유지:

5. 매 루프마다 현재 Todo 리스트의 상태를 시스템 프롬프트 또는 유저 피드백 상단에 재주입하여 컨텍스트 내 선택도를 유지합니다.

5.4.2 구조화된 처리 흐름

```
graph TD
    A[사용자의 모호하고 복잡한 요청] --> B[1단계: Todo 리스트 세분화 생성]
    B --> C[2단계: 첫 번째 Pending 항목 선택 및 실행]
    C --> D[3단계: 실행 결과 검증 및 Todo 상태 Completed 갱신]
    D --> E{모든 Todo 완료?}
    E -- 아니오 --> C
    E -- 예 --> F[최종 결과 종합 보고]
```

5.4.3 효과 및 검증

- **목표 이탈 방지**: 긴 태스크 수행 시 중간 단계 점검으로 신뢰성 대폭 상승
- **투명한 진행률 시각화**: 사용자가 에이전트의 현재 작업 단계를 명확히 파악 가능

[← 이전: v1 에이전트 모델](#) | [← 메인 로드맵으로 돌아가기](#) | [다음: v3 서브에이전트 메커니즘 →](#)

5.5 v3: 서브에이전트 메커니즘 (Subagent Mechanism)

독립된 컨텍스트 스코프와 자율 서브에이전트 위임

단일 메인 컨텍스트 윈도우에 대량의 로그나 검색 결과를 모두 채우면 **컨텍스트 오염(Context Poisoning)** 및 비용 폭증이 발생합니다.

v3 메커니즘은 주 에이전트(Main Agent)가 하위 탐색 작업을 담당하는 **서브에이전트(Subagent)**를 생성하고, 격리된 컨텍스트에서 탐색을 완료한 후 핵심 요약 결과만 수신하도록 합니다.

5.5.1 핵심 구조 및 동작 원리

```
sequenceDiagram
    participant Main as 메인 에이전트 (Main Context)
    participant Sub as 서브에이전트 (Isolated Context)
    participant Tools as 하위 도구 (Logs/Files/Searches)

    Main->>Sub: spawn_subagent(task_description)
    Note over Sub: 독립된 새 대화 히스토리 생성
    loop Subagent Loop
        Sub->>Tools: 탐색 및 로그 분석 실행
        Tools-->>Sub: 대용량 결과 반환
    end
    Note over Sub: 결과 요약 및 정제
    Sub-->>Main: 최종 요약 리포트 반환 (토큰 소모 최적화)
```

5.5.2 주요 이점 (Benefits)

- 메인 컨텍스트 보호:** 대용량 파티션 검색이나 빌드 로그가 메인 대화 윈도우를 오염시키지 않음
- 독립된 목표 집중:** 서브에이전트는 좁고 명확한 한 가지 목표에만 집중하여 정밀도 향상
- 병렬 탐색 확장성:** 여러 분야의 서브에이전트를 동시 구동 가능

[← 이전: v2 구조화된 계획 수립](#) | [← 메인 로드맵으로 돌아가기](#) | [다음: v4 스کیل 메커니즘 →](#)

5.6 v4: 스킬 메커니즘 (Skills Mechanism)

모듈식 지식 주입 및 런타임 동적 스킬 확장

에이전트에게 필요한 모든 도구와 지침을 시스템 프롬프트에 미리 다 넣으면 컨텍스트 토큰이 낭비되고 모델의 집중도가 떨어집니다.

v4 스킬 메커니즘은 특정 전문 작업(예: PDF 처리, 코드 리뷰, MCP 구축 등)이 필요할 때만 선언된 **스킬(Skills / SKILL.md)** 패키지를 동적으로 로드하고 주입하는 구조입니다.

5.6.1 스킬 파일 구조 (Agent Skills Spec)

```
skills/
├── agent-builder/
│   ├── SKILL.md (스킬 메타데이터 & 실행 지침)
│   ├── references/ (참조 가이드 문서)
│   └── scripts/ (자동화 헬퍼 스크립트)
├── code-review/
│   └── SKILL.md
└── pdf/
    └── SKILL.md
```

5.6.2 런타임 동작 절차

- 1. **스킬 감지**: 등록된 스킬 디렉토리의 YAML 메타데이터(`name` , `description`)를 읽어 기본 에이전트에 등록
- 2. **요청 매칭**: 사용자의 질문에 해당 스킬이 필요하면 `use_skill` 도구 호출
- 3. **지침 주입**: `SKILL.md` 문서의 세부 실행 절차 및 가이드를 현재 컨텍스트에 동적으로 결합 후 실행

5.6.3 결론 및 에이전트 루프 진화 요약

버전	핵심 메커니즘	해결하는 문제
v0	Bash 실행 & 재귀	최소 코드로 완벽한 기능 통합
v1	모델 기반 도구 선택	동적 Tool Call 및 관찰 피드백 루프
v2	Todo 리스트 계획 수립	긴 태스크 수행 시 목표 이탈(Drift) 방지

버전	핵심 메커니즘	해결하는 문제
v3	서브에이전트 위임	컨텍스트 오염 방지 및 탐색 분리
v4	동적 스킬 주입	런타임 모듈식 지식 확장

[← 이전: v3 서브에이전트 메커니즘](#) | [← 메인 로드맵으로 돌아가기](#)

6 스킬 & 레퍼런스

6.1 Agent Builder

Build AI agents for any domain - customer service, research, operations, creative work, or specialized business processes.

6.1.1 The Core Philosophy

The model already knows how to be an agent. Your job is to get out of the way.

An agent is not complex engineering. It's a simple loop that invites the model to act:

```
LOOP:  
  Model sees: context + available capabilities  
  Model decides: act or respond  
  If act: execute capability, add result, continue  
  If respond: return to user
```

That's it. The magic isn't in the code - it's in the model. Your code just provides the opportunity.

6.1.2 The Three Elements

1. Capabilities (What can it DO?)

Atomic actions the agent can perform: search, read, create, send, query, modify.

Design principle: Start with 3-5 capabilities. Add more only when the agent consistently fails because a capability is missing.

2. Knowledge (What does it KNOW?)

Domain expertise injected on-demand: policies, workflows, best practices, schemas.

Design principle: Make knowledge available, not mandatory. Load it when relevant, not upfront.

3. Context (What has happened?)

The conversation history - the thread connecting actions into coherent behavior.

Design principle: Context is precious. Isolate noisy subtasks. Truncate verbose outputs. Protect clarity.

6.1.3 Agent Design Thinking

Before building, understand:

- **Purpose:** What should this agent accomplish?
- **Domain:** What world does it operate in? (customer service, research, operations, creative...)
- **Capabilities:** What 3-5 actions are essential?
- **Knowledge:** What expertise does it need access to?
- **Trust:** What decisions can you delegate to the model?

CRITICAL: Trust the model. Don't over-engineer. Don't pre-specify workflows. Give it capabilities and let it reason.

6.1.4 Progressive Complexity

Start simple. Add complexity only when real usage reveals the need:

Level	What to add	When to add it
Basic	3-5 capabilities	Always start here
Planning	Progress tracking	Multi-step tasks lose coherence
Subagents	Isolated child agents	Exploration pollutes context
Skills	On-demand knowledge	Domain expertise needed

Most agents never need to go beyond Level 2.

6.1.5 Domain Examples

Business: CRM queries, email, calendar, approvals **Research:** Database search, document analysis, citations

Operations: Monitoring, tickets, notifications, escalation **Creative:** Asset generation, editing, collaboration, review

The pattern is universal. Only the capabilities change.

6.1.6 Key Principles

1. **The model IS the agent** - Code just runs the loop
2. **Capabilities enable** - What it CAN do
3. **Knowledge informs** - What it KNOWS how to do
4. **Constraints focus** - Limits create clarity
5. **Trust liberates** - Let the model reason
6. **Iteration reveals** - Start minimal, evolve from usage

6.1.7 Anti-Patterns

Pattern	Problem	Solution
Over-engineering	Complexity before need	Start simple
Too many capabilities	Model confusion	3-5 to start
Rigid workflows	Can't adapt	Let model decide
Front-loaded knowledge	Context bloat	Load on-demand
Micromanagement	Undercuts intelligence	Trust the model

6.1.8 Resources

Philosophy & Theory: - [references/agent-philosophy.md](#) - Deep dive into why agents work

Implementation: - [references/minimal-agent.py](#) - Complete working agent (~80 lines) - [references/tool-templates.py](#) - Capability definitions - [references/subagent-pattern.py](#) - Context isolation

Scaffolding: - [scripts/init_agent.py](#) - Generate new agent projects

6.1.9 The Agent Mindset

From: "How do I make the system do X?" **To:** "How do I enable the model to do X?"

From: "What's the workflow for this task?" **To:** "What capabilities would help accomplish this?"

The best agent code is almost boring. Simple loops. Clear capabilities. Clean context. The magic isn't in the code.

Give the model capabilities and knowledge. Trust it to figure out the rest.

6.2 The Philosophy of Agents

■ The model already knows how to be an agent. Your job is to get out of the way.

6.2.1 The Fundamental Insight

Strip away every framework, every library, every architectural pattern. What remains?

A loop. A model. An invitation to act.

The agent is not the code. The agent is the model itself - a vast neural network trained on humanity's collective problem-solving, reasoning, and tool use. The code merely provides the opportunity for the model to express its agency.

6.2.2 Why This Matters

Most agent implementations fail not from too little engineering, but from too much. They constrain. They prescribe. They second-guess the very intelligence they're trying to leverage.

Consider: The model has been trained on millions of examples of problem-solving. It has seen how experts approach complex tasks, how tools are used, how plans are formed and revised. This knowledge is already there, encoded in billions of parameters.

Your job is not to teach it how to think. Your job is to give it the means to act.

6.2.3 The Three Elements

1. Capabilities (Tools)

Capabilities answer: **What can the agent DO?**

They are the hands of the model - its ability to affect the world. Without capabilities, the model can only speak. With them, it can act.

The design principle: Each capability should be atomic, clear, and well-described. The model needs to understand what each capability does, but not how to use them in sequence - it will figure that out.

Common mistake: Too many capabilities. The model gets confused, starts using the wrong ones, or paralyzed by choice. Start with 3-5. Add more only when the model consistently fails to accomplish tasks because a capability is missing.

2. Knowledge (Skills)

Knowledge answers: **What does the agent KNOW?**

This is domain expertise - the specialized understanding that turns a general assistant into a domain expert. A customer service agent needs to know company policies. A research agent needs to know methodology. A creative agent needs to know style guidelines.

The design principle: Inject knowledge on-demand, not upfront. The model doesn't need to know everything at once - only what's relevant to the current task. Progressive disclosure preserves context for what matters.

Common mistake: Front-loading all possible knowledge into the system prompt. This wastes context, confuses the model, and makes every interaction expensive. Instead, make knowledge available but not mandatory.

3. Context (The Conversation)

Context is the memory of the interaction - what has been said, what has been tried, what has been learned. It's the thread that connects individual actions into coherent behavior.

The design principle: Context is precious. Protect it. Isolate subtasks that generate noise. Truncate outputs that exceed usefulness. Summarize when history grows long.

Common mistake: Letting context grow unbounded, filling it with exploration details, failed attempts, and verbose tool outputs. Eventually the model can't find the signal in the noise.

6.2.4 The Universal Pattern

Every effective agent - regardless of domain, framework, or implementation - follows the same pattern:

```
LOOP:
  Model sees: conversation history + available capabilities
  Model decides: act or respond
  If act: capability executed, result added to context, loop continues
  If respond: answer returned, loop ends
```

This is not a simplification. This is the actual architecture. Everything else is optimization.

6.2.5 Designing for Agency

Trust the Model

The most important principle: **trust the model**.

Don't try to anticipate every edge case. Don't build elaborate decision trees. Don't pre-specify the workflow.

The model is better at reasoning than any rule system you could write. Your conditional logic will fail on edge cases. The model will reason through them.

Give the model capabilities and knowledge. Let it figure out how to use them.

Constraints Enable

This seems paradoxical, but constraints don't limit agents - they focus them.

A todo list with "only one task in progress" forces sequential focus. A subagent with "read-only access" prevents accidental modifications. A response with "under 100 words" demands clarity.

The best constraints are those that prevent the model from getting lost, not those that micromanage its approach.

Progressive Complexity

Never build everything upfront.

```
Level 0: Model + one capability  
Level 1: Model + 3-5 capabilities  
Level 2: Model + capabilities + planning  
Level 3: Model + capabilities + planning + subagents  
Level 4: Model + capabilities + planning + subagents + skills
```



Start at the lowest level that might work. Move up only when real usage reveals the need. Most agents never need to go beyond Level 2.

6.2.6 The Agent Mindset

Building agents requires a shift in thinking:

From: "How do I make the system do X?" **To:** "How do I enable the model to do X?"

From: "What should happen when the user says Y?" **To:** "What capabilities would help address Y?"

From: "What's the workflow for this task?" **To:** "What does the model need to figure out the workflow?"

The best agent code is almost boring. Simple loops. Clear capability definitions. Clean context management. The magic isn't in the code - it's in the model.

6.2.7 Philosophical Foundations

The Model as Emergent Agent

Language models trained on human text have learned not just language, but patterns of thought. They've absorbed how humans approach problems, use tools, and accomplish goals. This is emergent agency - not programmed, but learned.

When you give a model capabilities, you're not teaching it to be an agent. You're giving it permission to express the agency it already has.

The Loop as Liberation

The agent loop is deceptively simple: get response, check for tool use, execute, repeat. But this simplicity is its power.

The loop doesn't constrain the model to particular sequences. It doesn't enforce specific workflows. It simply says: "You have capabilities. Use them as you see fit. I'll execute what you request and show you the results."

This is liberation, not limitation.

Capabilities as Expression

Each capability you provide is a form of expression for the model. "Read file" lets it see. "Write file" lets it create. "Search" lets it explore. "Send message" lets it communicate.

The art of agent design is choosing which forms of expression to enable. Too few, and the model is mute. Too many, and it speaks in tongues.

6.2.8 Conclusion

The agent is the model. The code is just the loop. Your job is to get out of the way.

Give the model clear capabilities. Make knowledge available when needed. Protect the context from noise. Trust the model to figure out the rest.

That's it. That's the philosophy.

Everything else is refinement.

6.3 Code Review Skill

You now have expertise in conducting comprehensive code reviews. Follow this structured approach:

6.3.1 Review Checklist

1. Security (Critical)

Check for: - ☐ **Injection vulnerabilities:** SQL, command, XSS, template injection - ☐ **Authentication issues:** Hardcoded credentials, weak auth - ☐ **Authorization flaws:** Missing access controls, IDOR - ☐ **Data exposure:** Sensitive data in logs, error messages - ☐ **Cryptography:** Weak algorithms, improper key management - ☐ **Dependencies:** Known vulnerabilities (check with `npm audit`, `pip-audit`)

```
# Quick security scans
npm audit           # Node.js
pip-audit           # Python
cargo audit         # Rust
grep -r "password\|secret\|api_key" --include="*.py" --include="*.js"
```

2. Correctness

Check for: - ☐ **Logic errors:** Off-by-one, null handling, edge cases - ☐ **Race conditions:** Concurrent access without synchronization - ☐ **Resource leaks:** Unclosed files, connections, memory - ☐ **Error handling:** Swallowed exceptions, missing error paths - ☐ **Type safety:** Implicit conversions, any types

3. Performance

Check for: - ☐ **N+1 queries:** Database calls in loops - ☐ **Memory issues:** Large allocations, retained references - ☐ **Blocking operations:** Sync I/O in async code - ☐ **Inefficient algorithms:** $O(n^2)$ when $O(n)$ possible - ☐ **Missing caching:** Repeated expensive computations

4. Maintainability

Check for: - ☐ **Naming:** Clear, consistent, descriptive - ☐ **Complexity:** Functions > 50 lines, deep nesting > 3 levels - ☐ **Duplication:** Copy-pasted code blocks - ☐ **Dead code:** Unused imports, unreachable branches - ☐ **Comments:** Outdated, redundant, or missing where needed

5. Testing

Check for: - ☐ **Coverage:** Critical paths tested - ☐ **Edge cases:** Null, empty, boundary values - ☐ **Mocking:** External dependencies isolated - ☐ **Assertions:** Meaningful, specific checks

6.3.2 Review Output Format

```
## Code Review: [file/component name]

### Summary
[1-2 sentence overview]

### Critical Issues
1. **[Issue]** (line X): [Description]
  - Impact: [What could go wrong]
  - Fix: [Suggested solution]

### Improvements
1. **[Suggestion]** (line X): [Description]

### Positive Notes
- [What was done well]

### Verdict
[ ] Ready to merge
[ ] Needs minor changes
[ ] Needs major revision
```

6.3.3 Common Patterns to Flag

Python

```
# Bad: SQL injection
cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")
# Good:
cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))

# Bad: Command injection
os.system(f"ls {user_input}")
# Good:
subprocess.run(["ls", user_input], check=True)

# Bad: Mutable default argument
def append(item, lst=[]): # Bug: shared mutable default
# Good:
def append(item, lst=None):
    lst = lst or []
```

JavaScript/TypeScript

```
// Bad: Prototype pollution
Object.assign(target, userInput)
// Good:
Object.assign(target, sanitize(userInput))

// Bad: eval usage
eval(userCode)
// Good: Never use eval with user input

// Bad: Callback hell
getData(x => process(x, y => save(y, z => done(z))))
// Good:
const data = await getData();
const processed = await process(data);
await save(processed);
```

6.3.4 Review Commands

```
# Show recent changes
git diff HEAD~5 --stat
git log --oneline -10

# Find potential issues
grep -rn "TODO\|FIXME\|HACK\|XXX" .
grep -rn "password\|secret\|token" . --include="*.py"

# Check complexity (Python)
pip install radon && radon cc . -a

# Check dependencies
npm outdated # Node
pip list --outdated # Python
```

6.3.5 Review Workflow

1. **Understand context:** Read PR description, linked issues
2. **Run the code:** Build, test, run locally if possible
3. **Read top-down:** Start with main entry points
4. **Check tests:** Are changes tested? Do tests pass?
5. **Security scan:** Run automated tools
6. **Manual review:** Use checklist above
7. **Write feedback:** Be specific, suggest fixes, be kind

6.4 MCP Server Building Skill

You now have expertise in building MCP (Model Context Protocol) servers. MCP enables Claude to interact with external services through a standardized protocol.

6.4.1 What is MCP?

MCP servers expose: - **Tools**: Functions Claude can call (like API endpoints) - **Resources**: Data Claude can read (like files or database records) - **Prompts**: Pre-built prompt templates

6.4.2 Quick Start: Python MCP Server

1. Project Setup

```
# Create project
mkdir my-mcp-server && cd my-mcp-server
python3 -m venv venv && source venv/bin/activate

# Install MCP SDK
pip install mcp
```



2. Basic Server Template

```
#!/usr/bin/env python3
"""my_server.py - A simple MCP server"""

from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

# Create server instance
server = Server("my-server")

# Define a tool
@server.tool()
async def hello(name: str) -> str:
    """Say hello to someone.

    Args:
        name: The name to greet
    """
    return f"Hello, {name}!"

@server.tool()
async def add_numbers(a: int, b: int) -> str:
    """Add two numbers together.

    Args:
        a: First number
        b: Second number
    """
    return str(a + b)

# Run server
async def main():
    async with stdio_server() as (read, write):
        await server.run(read, write)

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

3. Register with Claude

Add to `~/.claude/mcp.json`:

```
{
  "mcpServers": {
    "my-server": {
      "command": "python3",
      "args": ["/path/to/my_server.py"]
    }
  }
}
```

6.4.3 TypeScript MCP Server

1. Setup

```
mkdir my-mcp-server && cd my-mcp-server
npm init -y
npm install @modelcontextprotocol/sdk
```

2. Template

```
// src/index.ts
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";

const server = new Server({
  name: "my-server",
  version: "1.0.0",
});

// Define tools
server.setRequestHandler("tools/list", async () => ({
  tools: [
    {
      name: "hello",
      description: "Say hello to someone",
      inputSchema: {
        type: "object",
        properties: {
          name: { type: "string", description: "Name to greet" },
        },
        required: ["name"],
      },
    },
  ],
}));

server.setRequestHandler("tools/call", async (request) => {
  if (request.params.name === "hello") {
    const name = request.params.arguments.name;
    return { content: [{ type: "text", text: `Hello, ${name}!` }] };
  }
  throw new Error("Unknown tool");
});

// Start server
const transport = new StdioServerTransport();
server.connect(transport);
```

6.4.4 Advanced Patterns

External API Integration

```
import httpx
from mcp.server import Server

server = Server("weather-server")

@server.tool()
async def get_weather(city: str) -> str:
    """Get current weather for a city."""
    async with httpx.AsyncClient() as client:
        resp = await client.get(
            f"https://api.weatherapi.com/v1/current.json",
            params={"key": "YOUR_API_KEY", "q": city}
        )
        data = resp.json()
        return f"{city}: {data['current']['temp_c']}C, {data['current']['condition']['text']}"
```

Database Access

```
import sqlite3
from mcp.server import Server

server = Server("db-server")

@server.tool()
async def query_db(sql: str) -> str:
    """Execute a read-only SQL query."""
    if not sql.strip().upper().startswith("SELECT"):
        return "Error: Only SELECT queries allowed"

    conn = sqlite3.connect("data.db")
    cursor = conn.execute(sql)
    rows = cursor.fetchall()
    conn.close()
    return str(rows)
```

Resources (Read-only Data)

```
@server.resource("config://settings")
async def get_settings() -> str:
    """Application settings."""
    return open("settings.json").read()

@server.resource("file://{path}")
async def read_file(path: str) -> str:
    """Read a file from the workspace."""
    return open(path).read()
```

6.4.5 Testing

```
# Test with MCP Inspector
npx @anthropics/mcp-inspector python3 my_server.py

# Or send test messages directly
echo '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | python3 my_server.py
```



6.4.6 Best Practices

1. **Clear tool descriptions:** Claude uses these to decide when to call tools
2. **Input validation:** Always validate and sanitize inputs
3. **Error handling:** Return meaningful error messages
4. **Async by default:** Use `async/await` for I/O operations
5. **Security:** Never expose sensitive operations without auth
6. **Idempotency:** Tools should be safe to retry

6.5 PDF Processing Skill

You now have expertise in PDF manipulation. Follow these workflows:

6.5.1 Reading PDFs

Option 1: Quick text extraction (preferred)

```
# Using pdftotext (poppler-utils)
pdftotext input.pdf - # Output to stdout
pdftotext input.pdf output.txt # Output to file

# If pdftotext not available, try:
python3 -c "
import fitz # PyMuPDF
doc = fitz.open('input.pdf')
for page in doc:
    print(page.get_text())
"
```

Option 2: Page-by-page with metadata

```
import fitz # pip install pymupdf

doc = fitz.open("input.pdf")
print(f"Pages: {len(doc)}")
print(f"Metadata: {doc.metadata}")

for i, page in enumerate(doc):
    text = page.get_text()
    print(f"--- Page {i+1} ---")
    print(text)
```

6.5.2 Creating PDFs

Option 1: From Markdown (recommended)

```
# Using pandoc
pandoc input.md -o output.pdf

# With custom styling
pandoc input.md -o output.pdf --pdf-engine=xelatex -V geometry:margin=1in
```

Option 2: Programmatically

```
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

c = canvas.Canvas("output.pdf", pagesize=letter)
c.drawString(100, 750, "Hello, PDF!")
c.save()
```

Option 3: From HTML

```
# Using wkhtmltopdf
wkhtmltopdf input.html output.pdf

# Or with Python
python3 -c "
import pdfkit
pdfkit.from_file('input.html', 'output.pdf')
"
```

6.5.3 Merging PDFs

```
import fitz

result = fitz.open()
for pdf_path in ["file1.pdf", "file2.pdf", "file3.pdf"]:
    doc = fitz.open(pdf_path)
    result.insert_pdf(doc)
result.save("merged.pdf")
```

6.5.4 Splitting PDFs

```
import fitz

doc = fitz.open("input.pdf")
for i in range(len(doc)):
    single = fitz.open()
    single.insert_pdf(doc, from_page=i, to_page=i)
    single.save(f"page_{i+1}.pdf")
```

6.5.5 Key Libraries

Task	Library	Install
Read/Write/Merge	PyMuPDF	<code>pip install pymupdf</code>
Create from scratch	ReportLab	<code>pip install reportlab</code>

Task	Library	Install
HTML to PDF	pdftkit	<code>pip install pdfkit + wkhtmltopdf</code>
Text extraction	pdftotext	<code>brew install poppler / apt install poppler-utils</code>

6.5.6 Best Practices

1. **Always check if tools are installed** before using them
2. **Handle encoding issues** - PDFs may contain various character encodings
3. **Large PDFs:** Process page by page to avoid memory issues
4. **OCR for scanned PDFs:** Use `pytesseract` if text extraction returns empty